

Adaptive Learning Based Genetic Engineering Algorithm (ALGEA)

Majid Sohrabi (msohrabi@hse.ru)

December 2024

1 Introduction

In this project, we design an Adaptive Learning-based Genetic Engineering Algorithm (ALGEA) for combinatorial optimization problems. The main idea is to replace fixed probabilities of genetic operators with dynamically adjusted parameters that respond to the performance of the corresponding operators during the optimization process.

Five genetic operators are considered:

1. Crossover (λ_c),
2. Mutation (λ_m),
3. Crossover by Robust Chromosome (λ_{rc}),
4. Directed Mutation (λ_{dm}),
5. Gene Injection (λ_{gi}).

For each operator i , its probability λ_i is dynamically adjusted according to the quality of the offspring generated by that operator.

The fundamental principle is that an operator that consistently generates improved solutions should receive a reward and consequently have a higher probability in subsequent generations. Conversely, an operator that generates deteriorated solutions should receive a punishment and have its probability reduced.

Unlike a simple cumulative update based on the sum of all offspring improvements, the proposed adaptive mechanism explicitly separates positive and negative contributions. This allows the operator to receive a reward when its overall performance is positive and a punishment when its overall performance is negative.

Furthermore, the reward is gradually reduced as the optimization approaches its maximum iteration budget. This reflects the fact that, during the late stage of optimization, the population is expected to converge and increasingly large changes in operator probabilities are not desirable. Importantly, punishment remains active during this stage, allowing ineffective operators to continue losing probability.

Thus, the intended behavior of each adaptive parameter is to allow both increase and decrease during the main optimization process, while becoming increasingly conservative during the final stage of optimization.

2 Mathematical Formulation

For each genetic operator i , we define a dynamic hyperparameter

$$\lambda_i^t \tag{1}$$

representing the probability or rate of applying operator i at generation t .

The five adaptive parameters are

$$\lambda_c, \quad \lambda_m, \quad \lambda_{rc}, \quad \lambda_{dm}, \quad \lambda_{gi}. \tag{2}$$

Each parameter is constrained by lower and upper bounds:

$$\lambda_i^{\min} \leq \lambda_i^t \leq \lambda_i^{\max}. \quad (3)$$

The initial value is selected from the middle of the permitted range:

$$\lambda_i^0 = \frac{\lambda_i^{\min} + \lambda_i^{\max}}{2}. \quad (4)$$

Therefore, the adaptive mechanism starts without a strong preference toward either a high or low probability.

2.1 Notation

The main notation used in the proposed adaptive mechanism is given below.

1. λ_i^t : probability of operator i at generation t , where $i \in \{c, m, rc, dm, gi\}$.
2. $\Delta_{i,j}^t$: normalized improvement generated by offspring j of operator i at generation t .
3. α : learning rate controlling the magnitude of the adaptive update.
4. γ : proportion of the candidate solutions transferred from generation t to generation $t+1$ according to their improvement values.
5. $\lambda_i^{\min}$ and $\lambda_i^{\max}$: minimum and maximum allowed values of the probability of operator i .
6. $f(S)$: objective function value of solution S .
7. S_{old} : parent or reference solution used for evaluating the offspring.
8. S_{new} : offspring solution generated by operator i .
9. N_i^t : number of offspring generated by operator i at generation t .
10. ϵ : a small positive constant used to avoid division by zero.
11. T : maximum number of generations.

3 Performance Measurement

The performance of each operator is measured through the improvement of its generated offspring relative to the corresponding parent or reference solution.

For a *minimization* problem, the normalized improvement of offspring j generated by operator i is defined as

$$\Delta_{i,j}^t = \frac{f(S_{\text{old}}) - f(S_{\text{new}})}{|f(S_{\text{old}})| + \epsilon}. \quad (5)$$

Therefore,

$$\Delta_{i,j}^t > 0 \quad (6)$$

indicates that the offspring is better than the corresponding reference solution, whereas

$$\Delta_{i,j}^t < 0 \quad (7)$$

indicates that the offspring is worse than the corresponding reference solution.

For a maximization problem, the corresponding definition is

$$\Delta_{i,j}^t = \frac{f(S_{\text{new}}) - f(S_{\text{old}})}{|f(S_{\text{old}})| + \epsilon}. \quad (8)$$

Thus, in both cases,

$$\Delta_{i,j}^t > 0 \implies \text{improvement}, \quad (9)$$

and

$$\Delta_{i,j}^t < 0 \implies \text{deterioration.} \quad (10)$$

For the Crossover and Crossover by Robust Chromosome operators, which generate offspring from multiple parents, the reference objective value can be determined from the corresponding parent set.

For a minimization problem, the best parent objective value is

$$f(S_{\text{old}}) = \min_{k \in \mathcal{P}} f(S_{\text{parent},k}), \quad (11)$$

where $\mathcal{P}$ denotes the set of parents involved in generating the offspring.

For a maximization problem, the corresponding reference value is

$$f(S_{\text{old}}) = \max_{k \in \mathcal{P}} f(S_{\text{parent},k}). \quad (12)$$

This preserves the original performance measurement principle of the algorithm while allowing the same improvement measure to be applied to all five operators.

4 Positive Reward and Negative Punishment

For operator i at generation t , define the total positive improvement as

$$R_i^t = \sum_{j=1}^{N_i^t} \max(\Delta_{i,j}^t, 0). \quad (13)$$

Similarly, define the total deterioration magnitude as

$$Q_i^t = \sum_{j=1}^{N_i^t} \max(-\Delta_{i,j}^t, 0). \quad (14)$$

Thus,

$$R_i^t \geq 0 \quad (15)$$

represents the total reward obtained from successful offspring, while

$$Q_i^t \geq 0 \quad (16)$$

represents the total punishment associated with unsuccessful offspring.

This separation prevents positive and negative improvements from being directly combined into a single raw sum.

4.1 Normalized Operator Performance

Because the number of offspring generated by an operator may vary, the reward and punishment are normalized by their total magnitude.

The normalized reward component is defined as

$$r_i^t = \frac{R_i^t}{R_i^t + Q_i^t + \epsilon}. \quad (17)$$

Similarly, the normalized punishment component is

$$q_i^t = \frac{Q_i^t}{R_i^t + Q_i^t + \epsilon}. \quad (18)$$

Therefore,

$$0 \leq r_i^t \leq 1 \quad (19)$$

and

$$0 \leq q_i^t \leq 1. \quad (20)$$

Ignoring the very small ϵ ,

$$r_i^t + q_i^t \approx 1. \quad (21)$$

The difference between the two quantities provides a signed performance measure:

$$s_i^t = r_i^t - q_i^t. \quad (22)$$

Equivalently,

$$s_i^t = \frac{R_i^t - Q_i^t}{R_i^t + Q_i^t + \epsilon}. \quad (23)$$

Consequently,

$$-1 \leq s_i^t \leq 1. \quad (24)$$

The interpretation is straightforward:

$$s_i^t > 0 \implies \text{operator receives a positive performance signal,} \quad (25)$$

$$s_i^t < 0 \implies \text{operator receives a negative performance signal,} \quad (26)$$

and

$$s_i^t \approx 0 \implies \text{operator performance is approximately neutral.} \quad (27)$$

5 Illustration of the Reward–Punishment Measure

Consider an operator producing the following improvement values:

$$\Delta = \{0.02, 0.04, 0.01, -0.01, -0.005\}. \quad (28)$$

The positive reward is

$$R = 0.02 + 0.04 + 0.01 = 0.07, \quad (29)$$

and the punishment magnitude is

$$Q = 0.01 + 0.005 = 0.015. \quad (30)$$

Therefore,

$$s = \frac{0.07 - 0.015}{0.07 + 0.015} \approx 0.647. \quad (31)$$

The operator consequently receives a positive performance signal.
Now consider

$$\Delta = \{0.01, -0.02, -0.01, 0.005\}. \quad (32)$$

Then

$$R = 0.015 \quad (33)$$

and

$$Q = 0.03. \quad (34)$$

Therefore,

$$s = \frac{0.015 - 0.03}{0.015 + 0.03} = -0.333. \quad (35)$$

The operator consequently receives a negative performance signal.

Thus, the adaptive signal is determined by the relative balance between successful and unsuccessful contributions rather than by a raw sum of all improvement values.

6 Adaptive Learning Update

The basic adaptive probability update based on the signed performance measure is

$$\lambda_i^{t+1} = \lambda_i^t + \alpha s_i^t. \quad (36)$$

This equation preserves the original philosophy of the algorithm:

$$\text{good performance} \rightarrow \lambda_i \uparrow \quad (37)$$

and

$$\text{poor performance} \rightarrow \lambda_i \downarrow. \quad (38)$$

However, the signed performance score is bounded within the interval

$$[-1, 1], \quad (39)$$

which limits the magnitude of the adaptive signal.

7 Late-Stage Reward Attenuation

To obtain the desired behavior in which operator probabilities can increase and decrease during the main optimization process but become more conservative during the final stage, a progress-dependent reward factor is introduced.

Let T denote the maximum number of generations. The remaining optimization fraction is defined as

$$\rho_t = 1 - \frac{t}{T}. \quad (40)$$

At the beginning of optimization,

$$\rho_0 \approx 1, \quad (41)$$

whereas near the end,

$$\rho_T = 0. \quad (42)$$

The positive reward is multiplied by this remaining fraction, while the punishment remains fully active.

The resulting adaptive update is

$$\lambda_i^{t+1} = \lambda_i^t + \alpha [\rho_t r_i^t - q_i^t]. \quad (43)$$

This is the proposed final adaptive rule.

The equation can also be written directly in terms of R_i^t and Q_i^t :

$$\lambda_i^{t+1} = \lambda_i^t + \alpha \left[\rho_t \frac{R_i^t}{R_i^t + Q_i^t + \epsilon} - \frac{Q_i^t}{R_i^t + Q_i^t + \epsilon} \right]. \quad (44)$$

This formulation requires only a small modification to the original adaptive mechanism while introducing an explicit distinction between positive reward and negative punishment.

8 Interpretation of the Final Adaptive Rule

The proposed equation

$$\lambda_i^{t+1} = \lambda_i^t + \alpha [\rho_t r_i^t - q_i^t] \quad (45)$$

contains two competing terms.

The first term,

$$\rho_t r_i^t, \quad (46)$$

represents the attenuated reward received by the operator.

The second term,

$$q_i^t, \quad (47)$$

represents the punishment received by the operator.

Therefore, the change in the adaptive probability is

$$\Delta \lambda_i^t = \alpha [\rho_t r_i^t - q_i^t]. \quad (48)$$

If

$$\rho_t r_i^t > q_i^t, \quad (49)$$

then

$$\Delta \lambda_i^t > 0 \quad (50)$$

and the probability increases.

If

$$\rho_t r_i^t < q_i^t, \quad (51)$$

then

$$\Delta \lambda_i^t < 0 \quad (52)$$

and the probability decreases.

If

$$\rho_t r_i^t = q_i^t, \quad (53)$$

then

$$\Delta \lambda_i^t = 0 \quad (54)$$

and the probability remains unchanged before applying the bounds.

9 Behavior During Different Optimization Stages

The proposed update naturally produces different adaptive behavior at different stages of optimization.

9.1 Early Optimization

At the beginning of optimization,

$$\rho_t \approx 1. \quad (55)$$

Therefore, both reward and punishment have their full influence:

$$\Delta\lambda_i^t \approx \alpha(r_i^t - q_i^t). \quad (56)$$

If the operator performs well,

$$r_i^t > q_i^t, \quad (57)$$

and its probability increases.

If it performs poorly,

$$r_i^t < q_i^t, \quad (58)$$

and its probability decreases.

Therefore, the probability can move in either direction during the early stage.

9.2 Middle Optimization

During the middle stage,

$$0 < \rho_t < 1. \quad (59)$$

The reward gradually becomes more conservative, while punishment continues to operate.

Consequently, the operator probability can still increase if its positive performance is sufficiently strong, but the condition for an increase becomes progressively more difficult to satisfy.

This allows trajectories such as

$$0.50 \rightarrow 0.55 \rightarrow 0.51 \rightarrow 0.58 \rightarrow 0.54 \rightarrow 0.60. \quad (60)$$

The probability is therefore not forced to monotonically increase or decrease.

9.3 Late Optimization

Near the end of optimization,

$$\rho_t \rightarrow 0. \quad (61)$$

Therefore,

$$\lambda_i^{t+1} \approx \lambda_i^t - \alpha q_i^t. \quad (62)$$

Consequently, the positive reward becomes increasingly weak, while punishment remains active.

Thus, the late-stage behavior approaches a regime in which increases become difficult, while decreases remain possible.

10 Bounded Probability

After calculating the adaptive update, the probability is restricted to its permitted range:

$$\lambda_i^{t+1} = \min(\lambda_i^{\max}, \max(\lambda_i^{\min}, \lambda_i^t + \alpha[\rho_t r_i^t - q_i^t])). \quad (63)$$

Therefore,

$$\lambda_i^{\min} \leq \lambda_i^{t+1} \leq \lambda_i^{\max}. \quad (64)$$

No additional normalization of the five probabilities is required, because each λ_i represents an independent operator rate rather than a probability distribution whose values must sum to one.

11 Example of Adaptive Behavior

Assume

$$\lambda_i^{\min} = 0.10, \quad \lambda_i^{\max} = 0.90, \quad (65)$$

and therefore

$$\lambda_i^0 = 0.50. \quad (66)$$

Suppose

$$\alpha = 0.05. \quad (67)$$

During an early generation, assume

$$r_i^t = 0.75, \quad q_i^t = 0.25, \quad \rho_t = 1. \quad (68)$$

Then

$$\Delta\lambda_i^t = 0.05(0.75 - 0.25) = 0.025. \quad (69)$$

Therefore,

$$\lambda_i^{t+1} = 0.50 + 0.025 = 0.525. \quad (70)$$

The operator is rewarded.

Now suppose that in the next generation,

$$r_i^t = 0.30, \quad q_i^t = 0.70, \quad \rho_t = 1. \quad (71)$$

Then

$$\Delta\lambda_i^t = 0.05(0.30 - 0.70) = -0.020. \quad (72)$$

Therefore,

$$\lambda_i^{t+1} = 0.525 - 0.020 = 0.505. \quad (73)$$

The probability decreases.

This allows the desired dynamic behavior:

$$0.50 \rightarrow 0.525 \rightarrow 0.505. \quad (74)$$

If the operator becomes successful again, its probability can increase again during the main optimization stage.

12 Late-Stage Example

Suppose that near the end of the optimization,

$$\rho_t = 0.10. \quad (75)$$

Even if the operator performs well,

$$r_i^t = 0.80, \quad q_i^t = 0.20, \quad (76)$$

the adaptive change becomes

$$\Delta\lambda_i^t = \alpha(0.10 \times 0.80 - 0.20). \quad (77)$$

For

$$\alpha = 0.05, \quad (78)$$

we obtain

$$\Delta\lambda_i^t = 0.05(0.08 - 0.20) = -0.006. \quad (79)$$

Thus, even though the operator is still producing positive improvements, its probability decreases slightly because the optimization is approaching its final stage.

This behavior is intentional: the adaptive mechanism becomes increasingly conservative as the optimization approaches its termination budget. However, this is an optional part of the algorithm. We can propose it as another variant of the possible algorithm design.

13 Algorithm 1 Steps

13.1 Step 1: Initialization

1. Define the five adaptive operator probabilities:

$$\lambda_c, \lambda_m, \lambda_{rc}, \lambda_{dm}, \lambda_{gi}. \quad (80)$$

2. Define α as the learning rate.
3. Define $\lambda_i^{\min}$ and $\lambda_i^{\max}$ for each operator.
4. Initialize every operator probability at the midpoint of its permitted range:

$$\lambda_i^0 = \frac{\lambda_i^{\min} + \lambda_i^{\max}}{2}. \quad (81)$$

5. Generate the initial population.
6. Initialize the improvement values.

13.2 Step 2: Evolution Process

For each generation t , the following operations are performed.

1. Crossover

- (a) Apply crossover using λ_c^t .
- (b) Generate and evaluate the offspring.
- (c) Compute $\Delta_{c,j}^t$ for each offspring.
- (d) Calculate the positive reward R_c^t .
- (e) Calculate the punishment Q_c^t .
- (f) Calculate the normalized reward r_c^t and punishment q_c^t .
- (g) Calculate ρ_t .
- (h) Update λ_c^{t+1} using Eq. 63.

2. Mutation

- (a) Apply mutation using λ_m^t .
- (b) Generate and evaluate the offspring.
- (c) Compute $\Delta_{m,j}^t$.
- (d) Calculate R_m^t and Q_m^t .
- (e) Calculate r_m^t and q_m^t .
- (f) Update λ_m^{t+1} using Eq. 63.

3. Crossover by Robust Chromosome

- (a) Apply the robust-chromosome crossover using λ_{rc}^t .
- (b) Generate and evaluate the offspring.
- (c) Compute $\Delta_{rc,j}^t$.
- (d) Calculate R_{rc}^t and Q_{rc}^t .
- (e) Calculate r_{rc}^t and q_{rc}^t .
- (f) Update λ_{rc}^{t+1} using Eq. 63.

4. Directed Mutation

- (a) Apply directed mutation using λ_{dm}^t .
- (b) Generate and evaluate the offspring.
- (c) Compute $\Delta_{dm,j}^t$.
- (d) Calculate R_{dm}^t and Q_{dm}^t .
- (e) Calculate r_{dm}^t and q_{dm}^t .
- (f) Update λ_{dm}^{t+1} using Eq. 63.

5. Gene Injection

- (a) Apply gene injection using λ_{gi}^t .
- (b) Generate and evaluate the offspring.
- (c) Compute $\Delta_{gi,j}^t$.
- (d) Calculate R_{gi}^t and Q_{gi}^t .
- (e) Calculate r_{gi}^t and q_{gi}^t .
- (f) Update λ_{gi}^{t+1} using Eq. 63.

6. Selection for generation $t + 1$

- (a) Copy the best solution from each subpopulation ($Pop_t, Pop_{c_t}, Pop_{m_t}, Pop_{rc_t}, Pop_{dm_t}, Pop_{gi_t}$) to Pop_{t+1} .
- (b) Avoid duplicate chromosomes.
- (c) Construct a pool containing all remaining candidate chromosomes.
- (d) Sort the candidate chromosomes according to their improvement values Δ_j^t .
- (e) Transfer the best $\gamma\%$ of the candidate solutions according to Δ_j^t to Pop_{t+1} .
- (f) Select the remaining chromosomes using tournament selection until the population reaches the required size.
- (g) Continue until

$$|Pop_{t+1}| = Npop. \quad (82)$$

7. Check the stopping criterion.

8. If the stopping criterion is satisfied, terminate the algorithm; otherwise continue to generation $t + 1$.

Note: Duplicate chromosomes should not be transferred to the next generation.

14 Complete Adaptive Update for the Five Operators

For clarity, the five adaptive probabilities are updated according to the same general rule.

For crossover,

$$\lambda_c^{t+1} = \text{clip} [\lambda_c^t + \alpha (\rho_t r_c^t - q_c^t), \lambda_c^{\min}, \lambda_c^{\max}]. \quad (83)$$

For mutation,

$$\lambda_m^{t+1} = \text{clip} [\lambda_m^t + \alpha (\rho_t r_m^t - q_m^t), \lambda_m^{\min}, \lambda_m^{\max}]. \quad (84)$$

For robust-chromosome crossover,

$$\lambda_{rc}^{t+1} = \text{clip} [\lambda_{rc}^t + \alpha (\rho_t r_{rc}^t - q_{rc}^t), \lambda_{rc}^{\min}, \lambda_{rc}^{\max}]. \quad (85)$$

For directed mutation,

$$\lambda_{dm}^{t+1} = \text{clip} [\lambda_{dm}^t + \alpha (\rho_t r_{dm}^t - q_{dm}^t), \lambda_{dm}^{\min}, \lambda_{dm}^{\max}]. \quad (86)$$

For gene injection,

$$\lambda_{gi}^{t+1} = \text{clip} [\lambda_{gi}^t + \alpha (\rho_t r_{gi}^t - q_{gi}^t), \lambda_{gi}^{\min}, \lambda_{gi}^{\max}]. \quad (87)$$

Here,

$$\text{clip}(x, a, b) = \min(b, \max(a, x)). \quad (88)$$

15 Expected Adaptive Behavior

The proposed mechanism is designed to produce dynamic trajectories rather than monotonically decreasing probabilities.

For example, a possible trajectory for an operator may be

$$0.50 \rightarrow 0.53 \rightarrow 0.56 \rightarrow 0.54 \rightarrow 0.58 \rightarrow 0.55 \rightarrow 0.51 \rightarrow 0.48. \quad (89)$$

The increase and decrease are determined by the performance of the operator.

A successful operator can therefore receive several consecutive rewards:

$$\lambda_i^{t+1} > \lambda_i^t, \quad (90)$$

while poor performance produces

$$\lambda_i^{t+1} < \lambda_i^t. \quad (91)$$

As the optimization approaches its maximum generation,

$$\rho_t \rightarrow 0, \quad (92)$$

and the reward component gradually loses its influence.

Therefore, the final part of the trajectory can become predominantly decreasing:

$$0.58 \rightarrow 0.56 \rightarrow 0.55 \rightarrow 0.53 \rightarrow 0.51. \quad (93)$$

This behavior is consistent with the intended conservative adaptation during the final stage of the optimization process.

16 Summary of the Proposed Modification

The original adaptive rule was

$$\lambda_i^{t+1} = \lambda_i^t + \alpha \sum_{j=1}^{N_i^t} \Delta_{i,j}^t. \quad (94)$$

The revised rule replaces the raw sum with separate positive and negative contributions:

$$R_i^t = \sum_j \max(\Delta_{i,j}^t, 0), \quad (95)$$

and

$$Q_i^t = \sum_j \max(-\Delta_{i,j}^t, 0). \quad (96)$$

These are converted into normalized reward and punishment components:

$$r_i^t = \frac{R_i^t}{R_i^t + Q_i^t + \epsilon}, \quad (97)$$

and

$$q_i^t = \frac{Q_i^t}{R_i^t + Q_i^t + \epsilon}. \quad (98)$$

Finally, the adaptive probability is updated according to

$$\lambda_i^{t+1} = \text{clip} [\lambda_i^t + \alpha (\rho_t r_i^t - q_i^t), \lambda_i^{\min}, \lambda_i^{\max}], \quad (99)$$

where

$$\rho_t = 1 - \frac{t}{T}. \quad (100)$$

Therefore, the proposed mechanism preserves the original architecture of the algorithm while introducing an explicit reward–punishment mechanism and a progress-dependent attenuation of the reward.

The resulting adaptive behavior can be summarized as follows:

$$\text{Successful operator} \implies R_i^t \uparrow \implies r_i^t \uparrow \implies \lambda_i \uparrow, \quad (101)$$

whereas

$$\text{Unsuccessful operator} \implies Q_i^t \uparrow \implies q_i^t \uparrow \implies \lambda_i \downarrow. \quad (102)$$

As the optimization progresses,

$$\rho_t \downarrow, \quad (103)$$

which reduces the influence of positive reward while maintaining the punishment component.

Thus, the adaptive parameter can dynamically increase and decrease during the main optimization process, while becoming increasingly conservative as the optimization approaches its termination budget.

16.1 Experimental Setup for GQAP:

The list of the algorithms that are implemented in this study are presented in the following subsections with the comments. These algorithms need refinements before Tuning and Final Experiments, refer to the comments written in **red**:

16.1.1 Standard GA

Holland, J. H. (1992). Genetic algorithms. Scientific american, 267(1), 66-73.

The textbook simple genetic algorithm, kept as close to Holland’s original description as the capacity-constrained encoding allows. It deliberately does not use any of this project’s own enhancements below:

- Selection: plain fitness-proportionate (roulette wheel) selection on raw cost ($\text{fitness} = 1 / (\text{cost} + \epsilon)$) — no diversity weighting.
- Crossover: single-point crossover, applied with probability `crossover_rate` per mating pair; otherwise the pair passes through unchanged.
- Mutation: per-gene mutation with probability `mutation_rate` — each gene independently has a chance to be reassigned to a random facility.
- Replacement: full generational replacement (children entirely replace the population), with a minimal single-individual elitism (`elitism_count`, default 1) so the best-found solution is never lost to randomness — the one common, near-universal addition to Holland’s description.
- No diversity selection, no randomized repair sampling (uses deterministic GreedyRepair by default), no stagnation immigrants, no memetic local search.

This is the literal baseline the rest of the algorithms in this project are compared against.

Comments: Please, provide at least five discrete mutation operators, swap, big swap, insertion, reversion, random. At least, four different crossover operators, single-point, double-point, uniform-point, and random. Also, for applying Mutation on each chromosome we don’t iterate over the genes (or dimension of the chromosome) to see if we need mutation or not for every single gene because the complexity raises too much. You can find some of these operators via this link in Crossover and Mutation folders https://github.com/Majid-Sohrabi/GEA-GQAP/tree/main/Metaheuristics_GEA/Algorithm.

16.1.2 GEA

This project’s own enhanced GA. Where StandardGA above deliberately bypasses most of the ”Shared framework” section (it implements its own selection and replacement instead of calling into it), GEA is the opposite case: it doesn’t override or opt out of anything described there. Concretely, with fixed crossover and mutation rates:

- Selection: DiversitySelector (diversity-weighted roulette parents, elite + diversity/cost hybrid survivors — see ”Shared framework” above).
- Crossover / mutation: each generation draws `crossover_rate` $\times$ `population_size` crossovers (rounded to an even count) and `mutation_rate` $\times$ `population_size` mutations, each via `choose_crossover` / `choose_mutation`’s random dispatch.
- Repair: RFRRepair by default (randomized, diversity-preserving).
- Stagnation immigrants: if `best_solution` hasn’t improved for `stagnation_limit` iterations, `immigrant_rate` $\times$ `population_size` fresh random individuals are injected to give crossover new material to recombine with.
- Memetic local search: enabled (see ”Shared framework”).

Comments: Please, be sure that the same crossover and mutation operators have been used in the GA. Also, to be consistent with the original GEA make sure you have all these operators in this algorithm – Crossover, Mutation, Scenario 1: crossover with robust chromosome, Scenario 2: directed mutation, Scenario 3: gene injection. Because the goal here is not to develop a new GEA however to enhance the original one, so the original framework should be the same.

16.1.3 AdaptiveGEA

The self-adaptive variant proposed for this project: identical to GEA, except the crossover and mutation rates are no longer fixed — each is scaled by its own lambda multiplier (lambda_crossover, lambda_mutation, both starting at 1.0) that adapts every generation to how much improvement that operator actually produced:

1. Each offspring/mutant's improvement over its own parent baseline is normalized: $\text{delta} = (\text{parent_cost} - \text{child_cost}) / (\text{parent_cost} + \text{epsilon})$, and accumulated per operator (crossover sum, mutation sum).
Comparing to the parent baseline (not the global best) is a deliberate choice: comparing every child to the best-ever solution makes delta almost always negative once the population is decent, which would drive both lambdas toward lambda_min exactly as the run converges — choking off the exploration needed to escape a local optimum.
2. $\text{lambda_new} = \text{clip}(\text{lambda_old} + \alpha * \text{delta_sum}, \text{lambda_min}, \text{lambda_max})$.
3. Next generation's operator counts become $\text{base_rate} \times \text{population_size} \times \text{lambda}$ instead of just $\text{base_rate} \times \text{population_size}$.

An operator that's currently producing improvements gets used more; one that isn't gets throttled back, without any manual per-instance rate tuning.

Comments: IMPORTANT: This algorithm is not needed. It can be used in another project. This is out of the scope of this paper. We will propose this algorithm in another project with a new cost function after this paper.

16.1.4 Simulated Annealing (SA)

Bertsimas, D., & Tsitsiklis, J. (1993). Simulated annealing. *Statistical Science*, 8(1), 10–15.

Classic SA is based on a single solution; here the population is a bank of independent annealing chains (one walker per population slot) sharing a single global temperature schedule, reusing BaseGA's batched repair/evaluation instead of looping one permutation at a time in pure Python.

Each iteration, every walker proposes one neighbor via `choose_mutation` and:

- accepts it outright if it's better ($\text{cost} \leq \text{current}$), or
- accepts it with Metropolis probability $\exp(-\Delta/T)$ if it's worse,

then the shared temperature is cooled geometrically: $T \leftarrow \max(T_{\min}, T \times \text{cooling_rate})$. There is no crossover and no selection pressure between walkers — each chain only ever competes against its own previous state. Stagnation immigrants are still used (replacing random walkers, not added to a larger pool).

Comments: Do not use a population-based SA. It is not acceptable in the literature. I suggest changing your SA to the original single solution only or just simply remove this algorithm from the experiment. Please check the Dropbox folder project named "AL-GEA_New_Methaheuristic_Algorithm" then "Initial Codes for Algorithms we need" then "Simulated Annealing (SA)" for a sample code.

16.1.5 Particle Swarm Optimization (PSO)

Kennedy, J., & Eberhart, R. (1995, November). Particle swarm optimization. *Proceedings of ICNN'95* (Vol. 4, pp. 1942–1948).

There's no continuous position/velocity in a permutation encoding. Following the common discrete-PSO convention for permutation problems, each particle's "pull" is realized via existing operators instead of vector arithmetic:

- with probability `inertia_weight`: a random move via `choose_mutation` (exploration)
- with probability `cognitive_weight`: crossover with the particle's own personal best

- with probability `social_weight`: crossover with the swarm's global best

(the three weights are normalized to sum to 1). Each particle tracks its own position and personal best independently of self.population's ordering — `BaseGA.run()` re-sorts `self.population` by cost after every step, which would scramble a particle's identity if it were tracked by position in that list instead.

Comments: PSO is continuous originally. Here, the chromosome is integer. The encoding for the feasibility check must be clarified and justified. The discrete version of PSO also exist but you need to search and implement an appropriate version for discrete which is aligned with the original PSO. Please check the Dropbox folder project named "AL-GEA_New_Methaheuristic_Algorithm" then "Initial Codes for Algorithms we need" then "Particle Swarm Optimization (PSO)" for two samples code for PSO continious and binary optimizations.

16.1.6 Hybrid GA-PSO

Juang, C. F. (2004). A hybrid of genetic algorithm and particle swarm optimization for recurrent network design. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 34(2), 997–1006.

Each generation, the population is ranked by cost and split in two:

- the better-ranked `pso_fraction` undergo the PSO-style move described above (exploitation, pulled toward personal/global best)
- the rest are regenerated by ordinary GA crossover + mutation, selected from among themselves via `select_from_pool` (exploration)

An elitist replacement step then guarantees the swarm's best-ever solution survives into the next generation even if every particle's own move that round was a loss, as in the original paper.

Comments: Be sure the GA has the same operators as we have in other GA versions.

16.1.7 Hybrid GA-SA

Chen, P. H., & Shahandashti, S. M. (2009). Hybrid of genetic algorithm and simulated annealing for multiple project scheduling with multiple resource constraints. *Automation in Construction*, 18(4), 434–443.

GA crossover/mutation still supplies every candidate move; what changes is the acceptance rule. Each child must first pass a Metropolis test against its own parent baseline before it's even eligible to join the survivor pool: a child better than its baseline is always accepted, a worse one only gets a chance proportional to $\exp(-\Delta/T) \times T$. anneals down geometrically over the run, so early generations tolerate quality-losing moves (escaping local optima) while late generations behave like a plain elitist GA.

Comments: You can keep this algorithm. Remove the population-based SA (or change the SA to single solution in subsection 3.3.4). Be sure that GA operators are the same as the original one.

16.1.8 GEA_1

Modernization of GEA: replaces its randomly-dispatched crossover with a new operator, RC (Robust Chromosome) crossover — per gene, the child inherits from whichever parent's assigned facility has more remaining capacity slack (`cvar`), i.e. the more capacity-robust gene, independent of its assignment cost. This operator's rate is scaled by its own adaptive `lambda_rc` multiplier, using the exact same update/clip rule AdaptiveGEA uses for `lambda_crossover`:

1. apply RC crossover, evaluate offspring;
2. compute `delta_rc` against each offspring's parent baseline;
3. $\text{lambda_rc} \leftarrow \text{lambda_rc} + \alpha \times \text{delta_rc}$;
4. clip `lambda_rc` to $[\text{lambda_min}, \text{lambda_max}]$

Mutation stays GEA’s regular, non-adaptive mixed mutation operator.

Comments: IMPORTANT: this algorithm is out of scope (the same as subsection 3.3.3). This version should apply Crossover, Mutation, and Scenario 1: crossover with robust chromosome, all without adaptive operators. It should be implemented as original paper but as you provided enhancement in implementation and faster. We will propose the Adaptive Learning based version of GEA and its variants in the next project. Also, be sure that GA operators are the same as the original one (meaning the same crossover and mutation operators, it should be consistent).

16.1.9 GEA_2

Modernization of GEA: replaces its randomly-dispatched mutation with DM (Directed Mutation) — reuses mutation_greedy_reassign (move the single worst-assigned job to its cheapest feasible facility), a directed, fitness-improving move rather than a blind random one. Its rate is scaled by an adaptive lambda_dm multiplier, updated and clipped the same way as lambda_rc above. Crossover stays GEA’s regular, non-adaptive mixed crossover operator.

Comments: IMPORTANT: this algorithm is out of scope (the same as subsection 3.3.3). This version should apply Crossover, Mutation, and Scenario 2: directed mutation, all without adaptive operators. It should be implemented as original paper but as you provided enhancement in implementation and faster. We will propose the Adaptive Learning based version of GEA and its variants in the next project. Also, be sure that GA operators are the same as the original one (meaning the same crossover and mutation operators, it should be consistent).

16.1.10 GEA_3

Modernization of GEA: adds a third operator, GI (Gene Injection), alongside GEA’s regular fixed-rate crossover and mutation. GI reuses mutation_random (replace a handful of random genes with brand-new random facility assignments), injecting fresh genetic material rather than perturbing the existing assignment. Its own rate is scaled by an adaptive lambda_gi multiplier, updated and clipped the same way as lambda_rc and lambda_dm above.

Comments: IMPORTANT: this algorithm is out of scope (the same as subsection 3.3.3). This version should apply Crossover, Mutation, and Scenario 3: gene injection, all without adaptive operators. It should be implemented as original paper but as you provided enhancement in implementation and faster. We will propose the Adaptive Learning based version of GEA and its variants in the next project. Also, be sure that GA operators are the same as the original one (meaning the same crossover and mutation operators, it should be consistent).

16.1.11 Not implemented

GSAIS-KMeans, GSAIS-DBSCAN, and GSAIS-NN (Gromov, Sohrabi, & Fathollahi-Fard, Genetic Speciation Algorithm with Interplay among Species, under review at Algorithms) were requested but are not implemented: the paper is unpublished and not available, so there’s no reliable basis for a faithful implementation of its speciation/interplay mechanism.

Comments: Since these algorithms have not been published yet. It is okay at this stage, if they have not been implemented.

16.1.12 Hyperparameters Tuning

For each algorithm we need to define a three candidate values for each parameter and run for tuning based on the Taguchi table to find the best hyperparameters combination. These candidate values will be provided by Majid Sohrabi and based on the video instruction shared, you need to run the experiment for several test problems for GQAP and drive the RPD to find these hyperparameters. For tuning, we need to run each algorithm 30 times on one single test problem and the result considered would be the average of the 30 minimum found by the algorithm. Both table of parameters and RPD plots are needed in the paper.

16.1.13 Final Experiment

After finding the tuned hyperparameters then we need to run each algorithm on each test case 30 times and store the results. After that we will report the Min, Max, AVG, Std of these 30 runs of each algorithm for each test cases in GQAP. For this experiment, each algorithm should be allowed to search for 1000 seconds. After that, we can calculate the number of Best and the Number of Unique Best obtained by each algorithm and report it at the bottom of the main table of results (this statistic is based on the Min column).

16.1.14 Interval Plots on Std

By having the previous subsection results then we can drive the plot for the Std (30 runs) for all algorithms to check their robustness and accuracy.

16.1.15 Table of CPU time and Optimality GAP (OG)

We need to report the CPU time and OG (for small scale problems when we have the exact solution) in a table, and based on this table we can plot the CPU time for small scale and large scale separately. And also we can plot the OG as well.

16.1.16 Hitting Time Plot

We need to check the best run out of 30 runs and store it. Then try to find when was the first time the algorithm found the final minimum (which iteration and what seconds). In this case we can also make a plot or report it in a table.